MACHINE LEARNING PROJECT

FOREST FIRE RISK PREDICTION

Random Forest + Gradient-Boosted Trees

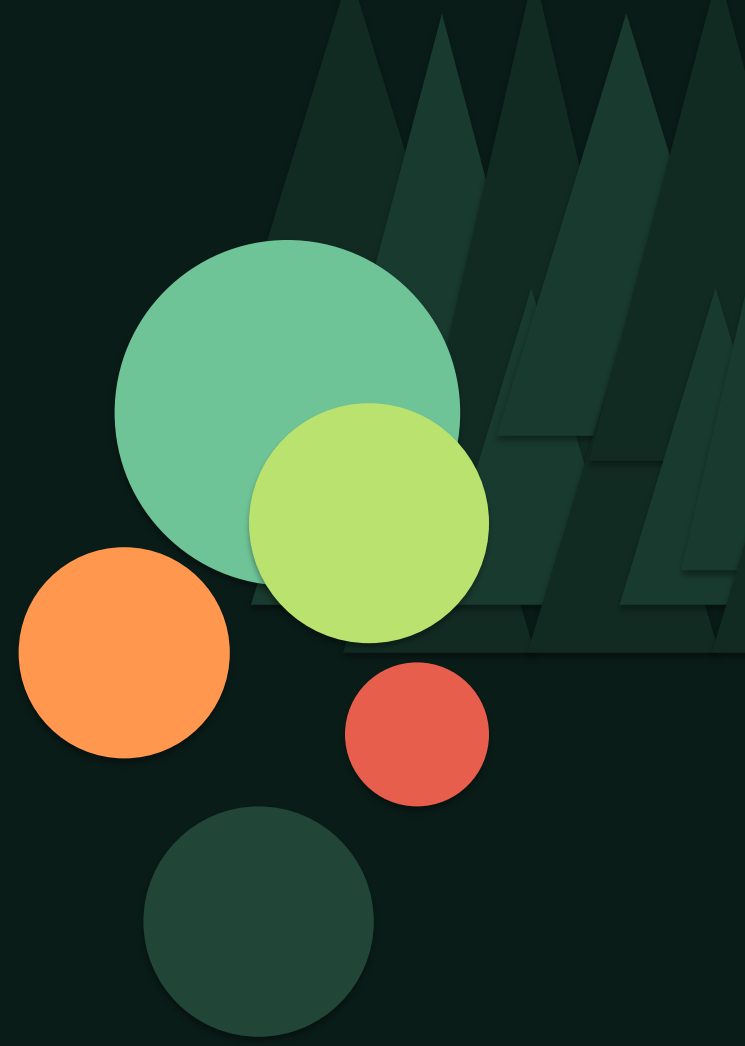
A data-driven approach to classifying fire-prone environmental conditions

L. Sainath • 2520090148

R. Nishanth • 2520090141

Guide: Dr. Y. Lakshmi Prasanna

Department of Computer Science & Information Technology



EARLY SIGNALS.

BETTER PREPAREDNESS.

Academic Year 2026–27

Why forest-fire risk prediction?

Forest fires can damage ecosystems, infrastructure and communities. Earlier risk screening supports preparedness.

THE DOMAIN

Weather and fuel conditions influence how easily a fire can ignite and spread.

THE CHALLENGE

Several variables interact non-linearly; simple thresholds may miss complex patterns.

THE OPPORTUNITY

Machine learning can learn patterns from historical fire-weather observations.

Project focus **Use meteorological + Fire Weather Index (FWI) features to classify conditions as FIRE or NOT FIRE.**

The problem we are solving

A practical screening task: convert a set of environmental readings into a consistent binary classification.

EXISTING APPROACH

Manual observation, expert judgement and fixed thresholds can be slow or inconsistent.

LIMITATIONS

Thresholds may not capture interactions among temperature, humidity, rain and FWI indicators.

WHO IS AFFECTED?

Forest departments, field teams, nearby communities and emergency planners.

Core question

Can historical weather and FWI data help identify conditions that resemble fire observations?

Why this project matters

A decision-support tool can help teams prioritise monitoring—not replace expert judgement.

REAL-WORLD NEED

Early awareness can support patrol planning,

ML MOTIVATION

Ensemble trees can model non-linear relatio

PROJECT MOTIVATION

Compare two strong tree ensembles, interpret thei

Guiding principle: a model prediction is a signal to review conditions, not a guarantee of a real-world event.

What the project aims to achieve

Five measurable objectives guide the work from raw data to a prediction prototype.

Prepare data 01

Clean records, validate fields and encode the target.

Explore patterns 02

Study distributions, class differences and correlations.

Train models 03

Fit Random Forest and Gradient Boosting fairly.

Evaluate 04

Compare classification metrics and 5-fold cross-validation.

Prototype 05

Save the selected model and return a class + probability estimate.

From manual checks to data-driven screening

This is the baseline context used to motivate the proposed ML workflow.

CURRENT PRACTICE

Observe weather, inspect vegetation/fuel conditions, and apply expert rules or fixed thresholds.

TECHNOLOGIES AROUND IT

Weather stations, field reports, fire indices and human expertise.

GAP ADDRESSED

A repeatable classifier can combine many variables at once and provide a consistent first-pass output.

Important: the project does not benchmark against a specific deployed agency system; this slide describes the general baseline described in the project motivation.

A compact prediction pipeline

The model turns user-supplied environmental readings into a Fire / Not Fire classification.

INPUT

Weather + FWI readings

PREPARE

Validate & order features

PREDICT

Saved Random Forest

PRESENT

Class + probability

Addresses the gap by making the prediction process repeatable, fast and easy to inspect.

System architecture / block diagram

Inputs are processed in the same feature order used during model training.

USER INPUT

Day, month, weather, FWI

VALIDATION

Types, ranges, ordering

ML MODEL

Random Forest .pkl

POST-PROCESS

Label + probability

RESULT

Fire / Not Fire

Training / evaluation path: dataset → cleaning → EDA → train/test split → CV + metrics → saved model

End-to-end workflow

The same split and evaluation approach was used to compare both classifiers.

01 Dataset cleaning

02 EDA & correlations

03 Stratified 80/20 split

04 Train RF + GB

05 Test metrics + 5-fold CV

06 Feature importance + ROC

07 Select RF + save model

09 / TECHNOLOGY

Tools and libraries

The modelling work was completed in Google Colab; VS Code is used for the frontend/backend prototype.

Python

Data preparation and model training

LANGUAGE

Google Colab

Experimentation and evaluation

NOTEBOOK

Pandas • NumPy

Cleaning, transformations and tables

DATA

Scikit-learn • Joblib

Ensemble models, metrics and model saving

ML

Matplotlib • Seaborn

EDA, correlation, ROC and importance plots

VISUALS

HTML • CSS/Tailwind • JavaScript

Frontend presentation; Python service loads the model

INTERFACE

System modules and responsibilities

Each module has a focused role in producing a prediction.

User interface

Collects the input readings

Data validation

Checks required values and feature order

Model loader

Loads the saved Random Forest artifact

Prediction engine

Runs predict() and predict_proba()

Result visualisation

Shows class, probability and a short explanation

Evaluation module

Stores metrics and comparison plots for reporting

Key implementation steps

The notebook experiments and saved model form the core of the prototype.

01 · Prepare	Remove formatting rows, clean labels and validate numeric columns.
02 · Split	Stratify target labels to preserve class proportions.
03 · Fit	Train Random Forest and Gradient Boosting with fixed random state.
04 · Evaluate	Generate classification metrics, confusion matrices, ROC and CV scores.
05 · Save	Persist the final Random Forest model and connect it to the prediction function.

The PPT uses the actual model metrics and plots; frontend screenshots can be replaced with real VS Code/browser captures when available.

What went into the model?

Prepared Algerian Forest Fires dataset used for binary classification.

244

Valid observations

after cleaning

12

Predictive features

Year excluded (constant)

138

Fire records

56.6% of dataset

106

Not Fire records

43.4% of dataset

WEATHER FEATURES

Day, Month, Temperature, Relative Humidity, Wind Speed, Rain

FWI FEATURES

FFMC, DMC, DC, ISI, BUI and FWI

Preprocessing: formatting/header rows removed; labels normalised; one malformed observation corrected; final data had no missing or duplicate rows.

Why compare Random Forest and Gradient Boosting?

Both are tree ensembles; they learn non-linear relationships without feature scaling.

RANDOM FOREST

- Trains many trees on bootstrap samples
- Combines their votes
- Often reduces variance and overfitting
- Selected as the final model

GRADIENT BOOSTING

- Adds trees sequentially
- Later trees focus on previous errors
- Captures subtle patterns
- Used as a strong comparison model

Main settings Random Forest: 100 trees, random_state=42 | Gradient Boosting: 100 estimators, learning_rate=0.1, max_depth=3

Held-out test performance

Random Forest correctly classified all 49 test observations in the selected split.

100%
Accuracy
Random Forest

100%
Precision
Random Forest

100%
Recall
Random Forest

100%
F1-score
Random Forest

1.000
ROC-AUC
Random Forest

GRADIENT BOOSTING

Accuracy 95.92% • Precision 93.33%
Recall 100% • F1 96.55% • ROC-AUC 0.9991

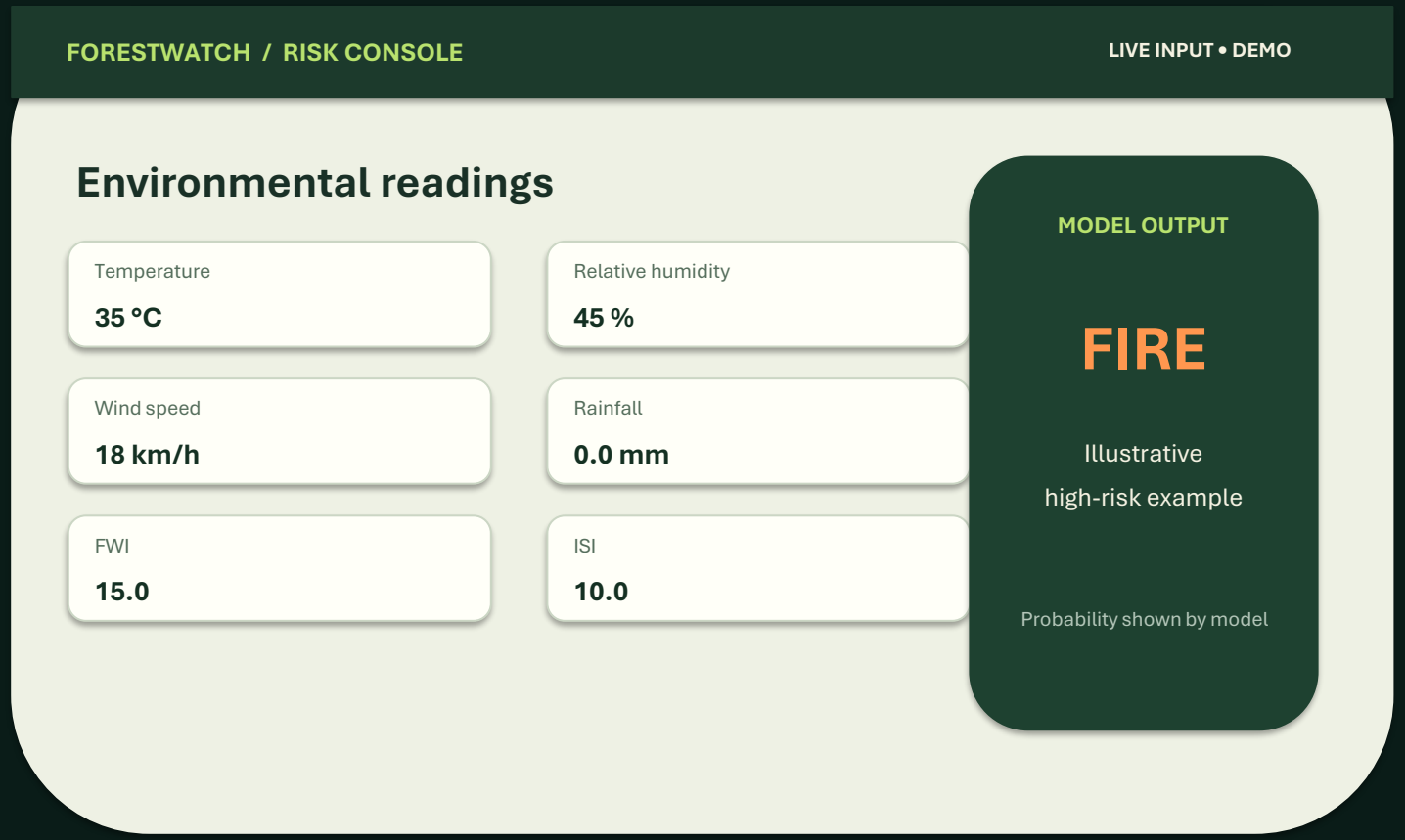
5-FOLD CROSS-VALIDATION

Random Forest: 97.94% ± 1.84%
Gradient Boosting: 97.14% ± 2.08%

Caution: 100% on one small test split does not guarantee perfect performance on new regions or seasons.

Prediction interface — concept

Replace this illustrative screen with a real browser screenshot after running the frontend.



HOME

Project purpose + risk context

INPUT

Weather and FWI measurements

RESULT

Class label + model estimate

What the proposed system improves

Benefits are strongest when the model is used as a consistent screening aid.

CONSISTENCY

Same input structure and model rules are applied for each prediction.

MULTI-VARIABLE

Combines weather and FWI indicators rather than relying on one threshold.

INTERPRETABLE OUTPUT

Metrics, confusion matrices and feature importance support model review.

Compared with a manual-only screening step, the prototype can produce a repeatable result in seconds.

Where could a tool like this help?

Potential applications of a validated and responsibly deployed version.

Forest departments

Prioritise patrols and situational checks

Fire management teams

Support readiness and resource planning

Environmental monitoring

Review patterns in weather and fuel indices

Research & education

Demonstrate applied ensemble classification

This prototype is not an official warning service and should not be used as the sole basis for emergency decisions.

What the current results do—and do not—say

Small dataset size and related FWI predictors require careful interpretation.

DATA LIMIT

244 observations from a limited regional/time setting; external validation is needed.

FEATURE OVERLAP

FWI variables are strongly correlated and some are closely linked to the target class.

PROBABILITY LIMIT

Prediction Probability is a model estimate, not a calibrated guarantee of actual fire occurrence.

The perfect Random Forest test score is encouraging, but can reflect the selected split and dataset structure.

How the project can grow

A roadmap from an academic prototype to a more robust decision-support system.

01

More data

Add more years, regions and seasons.

02

Validation

Test on unseen external datasets.

03

Calibration

Assess probability calibration and thresholds.

04

Live inputs

Integrate trusted weather feeds / sensor data.

05

Deployment

Add logging, monitoring and human review.

Prioritise external validation before using predictions in operational settings.

20 / CONCLUSION

A practical first step toward forest-fire risk screening.

KEY TAKEAWAYS

- Prepared 244 records and analysed 12 predictive features.
- Compared Random Forest with Gradient Boosting using test metrics and 5-fold CV.
- Random Forest achieved 97.94% mean cross-validation accuracy and was selected for the prototype.
- Built a workflow that returns Fire / Not Fire plus a model-estimated probability.

THANK YOU

Questions?

L. Sainath • R. Nishanth | CS&IT | Academic Year 2026–27